

Literature Review Sheet

Thesis: An Empirical Evaluation and Benchmark of AI-Generated Vulnerabilities in Code

Name: Eyad Hassan Abdelfatah

Paper	Main Features / Aspects	Method / Algorithm	Key Results	Gap Addressed	Gap Left (Limitation)	Link	My Comment
Sabra, Schmitt & Tyler (2025), Sonar (arXiv 2508.14727)	Quantitative static-analysis evaluation of 5 frontier LLMs (Claude Sonnet 4, Claude 3.7 Sonnet, GPT-4o, Llama 3.2 90B, OpenCoder-8B) on 4,442 Java tasks; tests whether functional pass rate correlates with code quality/security.	Code generated for 3 Java benchmark sources (MultiPL-E-mbpps, MultiPL-E-humaneval, ComplexCodeEval); functional Pass@1 measured on 2 of them; full SonarQube SonarWay ruleset (~550 rules) run on all generated code to classify bugs, vulnerabilities, and code smells by severity.	No correlation between functional pass rate and code quality — best model (Claude Sonnet 4, 77% pass) still averaged 2.11 issues per passing task; all models showed the same ~90–93% smells / 5–8% bugs / ~2% vulnerability split regardless of architecture/size; 56–70% of all vulnerabilities were BLOCKER/CRITICAL severity; newer Claude Sonnet 4 had more severe bugs/vulns than its predecessor despite a higher pass rate.	Refutes the assumption that better benchmark performance implies safer code, using today’s frontier commercial models (unlike CodeLM-Sec/SafeGenBench, which lean on older/open models).	Single language (Java) only; generic SonarQube rules, not CWE/CVE-mapped or exploit-verification; no adversarial/security-specific prompting; vulnerabilities are only ~2% of all issues found.	https://arxiv.org/abs/2508.14727	Broad code-quality study, not vulnerability-focused — vulnerabilities are buried under bugs/smells. Useful for the “pass rate ≠ security” motivating fact, but doesn’t go deep on why vulnerabilities happen or whether they’re exploitable. That gap is exactly where my thesis should sit.
Shahid, Ahmed & Ranjan (2025), LLM-CSEC (arXiv 2511.18966)	Vulnerability-focused evaluation of 10 LLMs on C/C++; each model given two personas — plain Code Generator (CG) vs. Secure Code Generator (SCG) told to write safe code — to test if security-focused prompting helps.	84 prompts built from MITRE CWE case scenarios (human+AI refined); code run through 3 SAST tools in parallel (CodeQL, Snyk Code, CodeShield); custom tool (SarifMiner) merges results and maps every CWE to real CVEs via NVD.	Every model produced critical vulnerabilities regardless of persona; the “secure” persona sometimes made things worse (CodeLlama’s SCG was more vulnerable than its CG); the 3 SAST tools barely agreed — CodeQL found 33 unique CWEs, Snyk 10, CodeShield 1, zero flagged by all three; models sometimes refused to generate code for security-sensitive prompts entirely.	First vulnerability-first benchmark with real CWE-to-CVE severity mapping; exposes that single-tool static analysis is unreliable by showing three tools barely overlap.	Static analysis only, no runtime/exploitability verification — CVE mapping shows a CWE class is historically dangerous, not that this exact snippet is exploitable; C/C++ only; no iterative prompt-refinement or CoT comparison tested.	https://arxiv.org/html/2511.18966v1	Strongest paper so far for motivating exploitability as a gap — their CVE mapping is still an indirect argument, not a direct one. Their 3-tool disagreement finding is strong ammunition for why single-tool static analysis (like Sabra et al.) isn’t trustworthy alone.
Li, Ding, Peng, Zhao, Gao, Gao & Gu (2025), SafeGenBench, ByteDance (arXiv 2506.05692)	Purpose-built benchmark: 558 test cases across 44 CWE types / 8 categories / 12 languages, derived from OWASP Top-10 + CWE Top-25; introduces a dual-judge framework (SAST + LLM-as-judge) instead of one detection method.	SAST-Judge (Semgrep) and LLM-Judge (DeepSeek-R1) each independently score code 0/1; a sample only counts as “secure” if both agree. Tested 13 frontier models (GPT-4o, o1, o3, Claude 3.5/3.7, Gemini 2.5 Pro, DeepSeek, Qwen3, Llama4) across zero-shot / zero-shot+safety-instruction / few-shot prompting.	Average zero-shot accuracy across 13 models was only 37.44%; explicit safety instructions raised accuracy 20+ points, few-shot added ~3% more; reasoning models consistently outperformed non-reasoning ones; SAST and LLM-judge only agreed code was secure in 61% of samples — LLM-judge caught 30% SAST missed, SAST caught 6% LLM missed, only 2.6% flagged by both.	Directly quantifies (not just asserts) that single-method detection is unreliable; broadest language/CWE coverage reviewed so far; strong template for multi-judge benchmark design.	Still detection-only, no exploitability verification (authors’ own stated limitation); single-function tasks only, no project-level generation; only one SAST tool and one LLM judge used; test prompts written in Chinese, a possible confound.	https://arxiv.org/abs/2506.05692	Closest methodological template to what I want to build, and the paper that most explicitly admits the same gap the other two circle: detection, however careful, is not proof of exploitability. Their dual-judge design is a good pattern to borrow for my own detection stage, before adding the exploit-confirmation layer they never attempt.

Paper	Main Features / Aspects	Method / Algorithm	Key Results	Gap Addressed	Gap Left (Limitation)	Link	My Comment
Yoo & Kim (2025), Security Analysis of Automated Code Generation (Tehnički Glasnik 19(4))	Hybrid static+dynamic security analysis of 20 ChatGPT-generated C programs (file processing, network, encryption, data processing) vs. 20 functionally-equivalent human-written programs.	Static analysis with Ghidra/IDA Pro/Objdump plus real runtime analysis with Valgrind (memory leaks) and Frida (runtime hooking/behavior); results mapped to OWASP Top 10 and a STRIDE threat model; two samples analyzed in depth with concrete attack scenarios built from the findings.	AI-generated code averaged 6.4% more vulnerabilities than human code (network security +18.8%, file ops +12.4%, error handling +12.4%); existing tools detected only 46.7% of AI-code vulnerabilities vs. 66.4% in human code; worked attack scenarios included 100%-accuracy key recovery of a fixed XOR encryption key from 3 known plaintext bytes, and a demonstrated memory-leak DoS path.	First paper reviewed to go beyond CWE-tagging into concrete, worked attack scenarios (key recovery, DoS, MITM, race-condition data loss) with explicit attacker steps and impact — the closest existing template to exploit confirmation.	Attack scenarios are illustrative case studies on only 2 hand-picked samples, not a systematic pipeline applied across all findings; no scripted/automated PoC framework; sample size only 20 programs total; ChatGPT only, C only (both flagged as their own limitations).	DOI link	Closest thing to a precedent for my exploit-confirmation idea, but they stop at hand-crafted illustrative examples rather than a repeatable, scaled methodology. My thesis can take their attack-scenario approach and make it systematic and automated across many samples/models, which is exactly the gap their own limitations section leaves open.

Main Research Gaps Identified

1. **Main gap — no verification of exploitability.** Sabra et al., LLM-CSEC, and SafeGenBench all stop at detection (generic static analysis; 3-tool SAST + CWE-to-CVE historical mapping; dual-judge SAST+LLM), but none confirm that a specific flagged vulnerability is actually exploitable in the generated code. Yoo & Kim come closest by building real attack scenarios, but only as hand-picked illustrative case studies on 2 samples, not a systematic pipeline. This is the gap my thesis will address: adding a scaled, scripted exploit-confirmation stage after detection for a scoped set of CWE categories (e.g., SQL injection, command injection, path traversal).
2. **Detection methods disagree with each other, and none of the papers treat this as solved.** LLM-CSEC: 3 SAST tools found zero common CWEs. SafeGenBench: SAST and LLM-judge only agreed 61% of the time. Yoo & Kim: existing tools miss 53.3% of AI-code vulnerabilities overall, with the gap especially large for dynamic tools. Sabra et al. used only one generic tool (SonarQube), the weakest setup of all four. This motivates using multiple detection methods (including dynamic analysis, not just static) as my own pipeline's first stage, before exploit confirmation.
3. **Vulnerabilities are usually studied as a side-category of broader code quality** (Sabra et al.: only ~2% of all findings were vulnerabilities, buried under bugs/smells) rather than as the primary object of study — a dedicated vulnerability-first benchmark is still comparatively rare.
4. **Security-focused prompting has inconsistent, sometimes counterproductive effects** (LLM-CSEC: “secure assistant” persona sometimes produced more vulnerable code than the plain assistant) — not something my thesis needs to solve, but worth acknowledging as a confound when designing prompts.
5. **Task scope stays small and narrow across all four studies** — single-function/single-file generation (no multi-file, project-level code), small sample sizes (Yoo & Kim: 20 programs), and single-model or single-language focus in some cases. Out of scope for a bachelor's thesis to fully solve, but worth naming explicitly as a boundary of my own study too.